

Credit Name: CSE 3130 Object-Oriented Programming 2.
Assignment Name: Account

The following questions can help you in thinking critically about your problem-solving processes:

Understanding the Problem:

How did you approach understanding the challenge?

- I read the assignment instructions and identified the important requirements such as inheritance, deposits, withdrawals, and account fees.
- I also checked the Account class before creating the child class.

Were there any parts of the problem you found confusing at first? If so, how did you resolve that confusion?

- I was confused about how to set the conditions for the extra charges for the personal and business accounts. I started to set the same shared method in Account to process the withdrawals and then added specific conditions inside BusinessAcct and PersonalAcct later.

Planning the Solution:

Did you create any or break the problem into smaller steps before coding?

- I break down the problem to the most minimize that I possibly have.

How did you decide on the tools, data structures, or algorithms to use?

- I break down the problem like this:
 - First I got the Customer class and expanded it based on the prompt.
 - Second, I got the sample of Account and updated the variables based on what I added in Customer and created basic steps for deposit and withdrawal.
 - Third, I would move the BusinessAcct and PersonalAcct and add the extra charge for the below threshold.
 - Then I add more details for easier use and test for errors.

Implementation:

Did you write the code in small or attempt the entire solution at first?

- I broke the question down in this order:
 - Customer:

```
public Customer(String fName, String lName, String st, String city1, String pro, String code)
{
    firstName = fName; //Sets the first name//
    lastName = lName; //Sets the last name//
    street = st; //Sets the street//
    city = city1; //Sets the city//
    province = pro; //Sets the province//
    postal = code; //Sets the postal code//
    //reflect the changes in the parameter
}
```

- I first declare what information needed to create an account.

```

//create changeCity method that asks the user
public void changeCity(String city1) {
    city = city1; //Updates the city to the user
}

public String getCity() {
    return city; //Returns the city
}

//create changeStreet method that asks the user
public void changeStreet(String st) {
    street = st; //Updates the street to the user
}

public String getStreet() {
    return street; //Returns the street
}

//create changeProvince method that asks the user
public void changeProvince(String pro) {
    province = pro; //Updates the province to the user
}

public String getProvince() {
    return province; //Returns the province
}

//create changePostalCode method that asks the user
public void changePostalCode(String code) {
    postal = code; //Updates the postal code to the user
}

public String getPostalCode() {
    return postal; //Returns the postal code
}
}

```

- Then I created smaller methods to record each information the computer receives when a user inserts their personal information.

```

public String toString() {
    String custString;
    //Update this string so that it contains the street, city, province, and postal code
    custString = " *First name: " + firstName + "\n"
        + " *Last name: " + lastName + "\n"
        + " *Street: " + street + "\n"
        + " *City: " + city + "\n"
        + " *Province: " + province + "\n"
        + " *Postal code: " + postal;
    return(custString); //Set the information for user to check after create new account//
}

```

- From there I created the temporary toString to present their information, which I test in client_test.

○ Account

```

public Account(double bal,String fName, String lName, String st, String city1, String pro, String postal1){
    {
        balance = bal; //Set the opening balance//
        cust = new Customer(fName, lName,st,city1,pro,postal1);//this constructor should reflect the new addition//
        //Create a new user using the provided information//
        acctID = fName.substring(0,1) + lName; //Build the user ID using first and last name//
    }
}

```

- Then I called all the information from the Customer.java (all the String values).
- I also asked for the opening balance when the user first creates their account and also generates their personal ID.

```

public double getBalance() {
    return(balance); //Return balance
}

/**
 * A deposit is made to the account.
 * pre: none
 * post: The balance has been increased by the amount.
 */
public void deposit(double amt) {
    balance += amt; //Update balance
}

```

- Then I added very basic algorithms for withdrawal and deposit.
- All the extra charge and withdrawal over their balance will be finished in their specific classes because the final balance will depend on how much the user uses their money.

```

public boolean equals(Object acct) {
    Account testAcct = (Account)acct;
    if (acctID.equals(testAcct.acctID))
        return(true); //Return true
    } else {
        return(false); //Return false
    }
}

```

- This class I created is used to verify if the account actually exists, or did the user enter the right account (using their given ID).

```

public String toString() {
    String accountString;
    NumberFormat money = NumberFormat.getCurrencyInstance();
    accountString = " " + acctID + "\n"; //Add ID to this version of
    accountString += cust.toString() + "\n"; //Paste the provided format
    accountString += " *Current balance is " + money.format(balance);
    return(accountString); //Return the string for user to verify their
    //If required higher security, this can be used
}

```

- Then I updated the toString method, this is the notification I will show the user the moment after they finish creating the account.

○ BusinessAcct/PersonalAcct:

```

public void deposit(double amt) {
    super.deposit(amt); //Use the
}

```

- Since the Account is BusinessAcct and PersonalAcct parent class, so that for deposit, I just call the method to update whenever the user deposits more money into the account.

```

public void withdrawal(double amt) {
    if (amt <= getBalance()) {
        super.withdrawal(amt); // use parent's withdrawal
        if (getBalance() < 100) {
            super.withdrawal(2); // charge extra $2 if the
        }
    } else {
        System.out.println("Not enough money in account.");
    }
}

```

- The same logic works for withdrawal, with some extra function: checking if the balance allows the withdrawal to happen or not (if the withdrawal is larger than the current balance)
- Also with the extra charge when the balance after withdrawal is lower than the threshold (Personal: \$100 charges \$2/ Business: \$500 charges \$10)

○ Client_test:

```

PersonalAcct personal = new PersonalAcct(1000,"Matthew","James","159 Macewan Valley","Calgary","Alberta","H6
BusinessAcct business = new BusinessAcct(10000,"Quang","Nguyen","123 HauptstraÙe","Charlottenburg","Berlin",

```

- I created two clone account to test if the entire code from 4 classes or not (as a final test)

```

System.out.println("-----");
System.out.println("Menu.\n" +
    "1. Create New Personal Account.\n" +
    "2. Create New Business Account.\n" +
    "3. Deposit.\n" +
    "4. Withdraw.\n" +
    "5. Cancel.\n");
System.out.print(" "); //Reads the user's menu choice//
decision = userInput.next();

```

- And I looped the entire procedure that allows the user to continually work with the bank with their business and personal accounts until they are finished.
- To finish the work, user can choose 5 and the loop will break

How did you test your solution along the way to make sure it was working?

- I broke the question down to smaller attempts, by doing each class one by one and test each class before I move on so that I do not get stuck with any bugs later on.

Overcoming Challenges:

What part of the problem was the most difficult for you?

- There was a tricky part in the withdrawal method in the Account.java class.
- Initially, I created a method to go over when the balance reaches 0 and prevent the user from withdrawing further.
- However, with this method, the extra charge cannot work when the user withdraws their entire account, with the charge will be counted $\Rightarrow$ lead to the balance to be negative, which the method in Account will prevent it from happening.

Learning:

Was there anything you learned that you think will help you with future challenges?

- I attempted this project first, and it helped me a lot in how to manage multiple classes together and also how to minimize the method I have to use in the future project while maintaining the output of the project.